



Evolutionary Echo State Network: A neuroevolutionary framework for time series prediction

Sebastián Basterrech, Gerardo Rubino

► To cite this version:

Sebastián Basterrech, Gerardo Rubino. Evolutionary Echo State Network: A neuroevolutionary framework for time series prediction. Applied Soft Computing, 2023, 144, pp.110463. <10.1016/j.asoc.2023.110463>. <hal-04389423v2>

HAL Id: hal-04389423

<https://inria.hal.science/hal-04389423v2>

Submitted on 11 Jan 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Copyright - All rights reserved

Evolutionary Echo State Network: a neuroevolutionary framework for time series prediction

Sebastián Basterrech^a, Gerardo Rubino^b

^a*Department of Computer Science, Faculty of Electrical Engineering and Computer Science, VSB-Technical University of Ostrava, 17. listopadu 2172/15, Ostrava, 70833, Czech Republic*

^b*INRIA Rennes–Bretagne Atlantique, Rennes, France*

Abstract

From one side, Evolutionary Algorithms have enabled enormous progress over the last years in the optimization field. They have been applied to a variety of problems, including optimization of Neural Networks' architectures. On the other side, the Echo State Network (ESN) model has become increasingly popular in time series prediction, for instance when modeling chaotic sequences. The network has numerous hidden neurons forming a recurrent topology, so-called *reservoir*, which is fixed during the learning process. Initial reservoir design has mostly been made by human experts; as a consequence, it is prone to errors and bias, and it is a time consuming task.

In this paper, we introduce an automatic general neuroevolutionary framework for ESNs, on which we develop a computational tool for evolving reservoirs, called EVOLUTIONARY Echo State Network (EvoESN). To increase efficiency, we represent the large matrix of reservoir weights in the Fourier space, where we perform the evolutionary search strategy. This frequency space has major advantages compared with the original weight space. After updating the Fourier coefficients, we go back to the weight space and perform a conventional training phase for full setting the reservoir architecture. We analyze the evolutionary search employing genetic algorithms and particle swarm optimization, obtaining promising results with the latter over three well-known chaotic time series. The proposed framework leads fast to very good results compared with modern ESN models. Hence, this contribution positions an important family of recurrent systems in the promising neuroevolutionary domain.

Keywords: Neuroevolution, Echo State Networks, Evolutionary

1. Introduction

Recurrent network topologies can be used to incorporate temporal information in the training of Neural Networks (NNs). They are parametric dynamical systems with abilities for learning and memorizing input sequences [1, 2]. However, Recurrent Neural Networks (RNNs) have often come at the expense of much longer training times. They are complex and difficult to be properly trained, particularly when the network is large, and when it is necessary to learn long-term data dependencies [3]. Since 2001, a type of RNNs has been investigated under the name of Reservoir Computing (RC), and it has captured considerable attention due to its outstanding results for modeling chaotic systems and in solving real-world problems [2, 4, 5]. RC models overcome the important limitation of training procedures while introducing no significant disadvantages for time series processing applications. In spite of its simplicity, classic RC methods sometimes outperformed on certain tasks other well-established recurrent systems as Long Short-term Memory (LSTM) networks [6]. The RC paradigm has been developed from three main concepts [2, 7]: Liquid State Machine (LSM), Echo State Network (ESN) and Backpropagation-decorrelation (BDPC). These three models share similar concepts regarding both the network design and the training approach. They have a nonlinear parametric dynamical system (i.e. an RNN with fixed weights), called *reservoir*, which has two main roles: feature extraction and memorization of input sequence. The learning protocol is restricted to adjust a simple parametric function of the output weights only, leading to a fast training procedure. The readout structure, i.e. the mapping between the extracted features at the reservoir to the output, is memory-free (often just a linear regression). The approach relies on the assumption that the reservoir is able to memorize historical data, and the readout structure is enough for designing an efficient learning model.

Recent developments in machine learning have also provoked a rising interest in evolutionary algorithms (EAs) as optimization tools for NNs [8]. Evolutionary optimization techniques have also been applied for optimizing recurrent architectures. Many early works have used EAs for finding optimal RC hyperparameters [2, 9, 10, 11]. Likewise, EAs have also been employed

for evolving recurrent topologies. A combination of Particle Swarm Optimization (PSO) and Extreme Learning Machine (ELM) was studied in [12], where the optimization is done over a small region of the searching space. Following the ELM and RC principles, the rest of the weights are fixed and don't participate in the network tuning. Recently, Neat and HyperNEAT methods were used for evolving the reservoirs in [13, 14]. In addition, EAs have also been employed for optimizing the hyperparameters of multi-layered stacked reservoirs, which are models with reservoirs organized in a cascade scheme [15, 16, 17].

Even though RC methods have accomplished very good performances and wide recognition, the RC area has also received some criticisms [1, 3]. The initial design of reservoirs is still often made by experience. Sometimes, even *brute force* is used, by exploring an arbitrary pre-defined region of the hyperparameters' space [18, 19]. Thus, it is computational expensive and requires numerous trails. Another known drawback is that different reservoirs with the same hyperparameters may produce significantly different results [20]. In addition, since the weights in the reservoir are initialized and remain fixed during the training process, the computational power of the recurrences is likely to be not fully harnessed [3, 21]. The model relies on the non-linear dynamics presented in the fixed initialized reservoir [3].

Motivated by the above discussion, we aim to improve the potential of large randomly initialized and non-adaptable reservoirs using EAs. Genetic encoding is a significant decision at the moment of the EA design. Specially, if the optimization is made over large complex structures. An efficient indirect encoding for NNs was developed in [22]. The weights were represented using the coefficients of the Discrete Cosine Transform (DCT). The evolutionary search was conducted in the frequency-domain instead of in the weight space. A first glimpse into evolving reservoirs in the frequency domain was developed in our previous conference article [23], where the EVolutionary Echo State Network (EvoESN) model was introduced. The EvoESN model is inspired by the works [1, 24], where the authors compress the network in the frequency domain.

None of the above-mentioned papers that apply EAs in the RC area have worked in the Fourier space, they apply direct encodings that present limitations for optimizing large networks. A majority of the works apply EAs to the optimization of the hyperparameters, or to adjust the weights among hierarchical reservoirs. To the best of our knowledge, the EvoESN is the first method that combines the potential of the Fourier compression with

the RC area, and fine-tunes the reservoir weights, avoiding the problems of the gradient-based methods.

The purpose of this paper is to improve the EvoENS model by means of a better EA component, to provide a deeper analysis of the model, and to analyze the evolutionary phase and the reservoir stability. We make the following threefold contributions:

- (i) **Framework for evolving reservoirs in the frequency domain.** It is well known that the initialization of the recurrent weights impacts on the model’s performance. Furthermore, it seems that to have fixed network weights (independently of the input data) does not totally employ the power of recurrent system. We study the integration of EAs for evolving the reservoirs in a compressed domain, and compare them with baseline modern ESN solutions.
- (ii) **Exploration characteristics and stability control.** We present a systematic analysis of the framework hyperparameters. Furthermore, we analyze the evolution of the feasible solutions according to the main parameter related to the recurrences’ stability. We also establish a more general control framework that includes an analysis regarding the initialization of the EA population.
- (iii) **Search strategy analysis.** We introduce a more exhaustive machine learning study. In our previous work [25], the fine-tuning of the reservoir weights was done using Genetic Algorithms (GAs). In this paper, we also discuss the improvement obtained by the use of the PSO algorithm. PSO is a population-based optimization method well-known for its achieved good results in problems with continuous and high-dimensional fitness landscape [26, 27].

The experimental results show the improvements obtained by the new EvoESN with respect to the initial version. The method finds well-performing reservoirs quickly, and achieves also significant improvements compared with other state-of-the-art recurrent networks. Experiments were made over two widely-investigated chaotic dynamical systems: the Mackey Glass system and the Lorenz system [1, 18, 28, 29, 30, 31, 32]. In addition, we present results over a real-world data set entitled Sunspot series [33], that has been also widely studied in the RC domain.

The outline of the article is the following. In the next section, we briefly describe the necessary background related to recurrent systems, and review previous work about EAs over RNNs. Section 3 introduces the ESN model, its properties and modern variations. The main contribution (the EvoESN model) is presented in Section 4. Experimental results are analyzed in Section 5. A final discussion provides a summary of the limitations and advantages of the framework presented in this paper, and discuss future research directions.

2. Preliminaries

2.1. Problem characterization

Recurrent systems have proven to be effective for solving learning problems where the ordering of data matters, such as time-series forecasting, time-series classification, sequence prediction and generative modeling. Those tasks are often grouped under the global concept of Temporal Learning [2], where the learning data is composed of dependent instances, e.g. $\mathbf{u}(t)$ depends on $\mathbf{u}(t \pm \delta)$, $\delta \leq \Delta$, for some non-large Δ value. In contrast, non-temporal problems have instances that are independent of each other.

Let us consider a supervised context where the dataset is composed of N input-output pairs $(\mathbf{u}, \mathbf{y})$; the input space has dimension n and the output space has dimension o . For simplicity, we consider sequences indexed in discrete spaces. The goal in Temporal Learning, that is the most common application of ESNs, is to predict a future value of a sequence (or input signal) given the present value and some values in the past; for instance, given a sequence $\mathbf{u}(t-k), \dots, \mathbf{u}(t-1), \mathbf{u}(t)$, the goal could be to predict the output $\mathbf{y}(t+\delta)$, with $\delta \geq 0$.

The learning process has an initial training phase where the network is driven by external data, and the network weights are trained solving an optimization problem. Then, an autonomous phase is performed where the input instances are provided by the model itself using previous predicted values [34]. In the domain of time series prediction, the autonomous phase often appears under the name of free-run prediction.

2.2. Motivations

There are many variations of RNN architectures, where the recurrences have another form; for more information we suggest [6]. However, training

recurrent systems is still difficult [6, 18]. Classic RNNs, LSTMs or Transformers can be expensive to train, and often requires numerous samples to reach a satisfactory accuracy [35]. Moreover, learning algorithms based on gradient and curvature information for training recurrent networks have well-identified technical issues [3, 36]. For instance, first-order algorithms (based on the gradient information) may suffer from the vanishing and exploding gradient problem [36]. Second order techniques (based on the curvature information) may present scalability issues; for some of them, there is a quadratic relationship between the size of the Hessian matrix and the number of weights [3]. Large networks have additional inconveniences, the most common mentioned in the literature are: not guaranteed convergence, large training times, and storage problems [2, 6, 35]. The RC paradigm has emerged as a feasible option of using recurrent networks, avoiding the well-known issues on the optimization of the recurrent networks [2].

2.3. Neuroevolution: combining the power of neural nets with evolutionary algorithms

Evolutionary computation algorithms have been used for a long time for evolving architectures as well as for finding the network weights [8, 37]. Neuroevolution is a nature-inspired computational-matematical model that combines the potential of both NNs with EAs [38]. Evolutionary approaches in specific types of large-scale problems outperform gradient descent optimization [39]. Recently, more evolutionary techniques have been developed, specifically for optimizing large NNs [1, 40, 41]. Neuroevolution provides a general framework for the joint optimization of the network structure, the topology, and the parameters related to activation function and other characteristics of the network [41, 42]. There is a wide variety of evolutionary approaches in the neuroevolution field [37, 38]. From a global viewpoint, the network structure and weights can be evolved either simultaneously or separately. Looking at the network components, every neuron can be either optimized independently or complete layers can be optimized at the same logical time. A breaking point in the neuroevolutionary research was the introduction of the neuroevolution of augmenting topologies (NEAT) method [41]. The technique starts with a simple architecture and increases the edge connections over time. It has obtained good performances in embedded systems.

2.4. Encoding network architectures

A recurrent topology may produce a significant impact on the model’s performance [43, 16]. Moreover, the evolution of recurrent systems has additional difficulties. A recurrent structure (formally, a dynamical system) must manage the additional temporal parameter, and the space-complexity of the encoding is really challenging [39, 44]. The architecture design predominantly remains driven by randomized and heuristic approaches [43]. Another relevant problem in neuroevolution refers to the evolutionary process itself: how to combine single individuals? [37]. Naive crossover operations are not really suitable for combining two recurrent structures [37, 45], they can even impact negatively on the model’s performance. Some authors directly avoid crossovers and pay attention only to the selection process [37]. This fact can be attributed to several reasons such as the invariance of neurons to permutation (i.e., the *competing convention* problem) [46], the semantic correctness of the operations, the case of large search spaces, and the high sensitivity issues (small perturbations on single networks can produce significant different results) [45].

Some of the before mentioned issues have been tackled by recent advances in the field of neuroevolution. Controlled weight perturbation using output gradient information was introduced in [45]. The approach effectively increases the ability of even simple genetic algorithms to evolve neural structures. Similarly, safe operations (controlled crossover operations) were introduced in [46]. Augmenting topologies has produced an important advance in the area. However, a significant drawback is that it uses a direct encoding, implying that it is not scalable to large networks [1, 37]. Overall, neuroevolution over deep RNNs has become a hot research topic [39, 44]. When using a direct encoding (the candidate solution represents the entire network), traditional algorithms such as GNARL or NEAT struggle with the complexity of the problem [37, 41]. Although modified and indirect methods such as HyperNEAT [42] and coDeepNeat [47] overcome some of these initial limitations, new methods are required to more efficiently encode and evolve large recurrent architectures, in order to solve large-scale real-world problems [39, 44, 48]. In Section 4.3 we discuss a recent introduced indirect encoding based on Fourier transforms, which is the one applied in our EvoESN model.

3. Echo State Networks

3.1. Model specification

We first formalize the standard ESN model that probably is still the most popular in the RC area. Let's consider a network with n input neurons, p hidden neurons and o output neurons. The dynamics of the model, as a function of the discrete time variable t , has the form [49]:

$$\mathbf{x}(t) = f(\mathbf{W}^{\text{in}}\mathbf{u}(t) + \mathbf{W}^{\text{h}}\mathbf{x}(t-1)), \quad (1)$$

where $\mathbf{W}^{\text{in}}$ is a $p \times n$ matrix with the input-to-hidden weights and the $p \times p$ matrix $\mathbf{W}^{\text{h}}$ has the hidden-to-hidden weights (reservoir weights). The model output $\mathbf{y}(t) \in \mathbb{R}^o$ is computed by the readout layer

$$\mathbf{y}(t) = g(\mathbf{W}^{\text{out}}[\mathbf{u}(t); \mathbf{x}(t)]), \quad (2)$$

where $\mathbf{W}^{\text{h}}$ is the reservoir-to-output weight matrix, $g(\cdot)$ is a predefined coordinate-wise function and $[\cdot; \cdot]$ denotes vector concatenation. For simplicity, bias terms on previous expressions are omitted on the equations, a common practice since it is straightforward to implement them by adding dummy terms. The transition function $f(\cdot)$ must satisfy some properties in order to perform stable recurrent updates. The function should be 1-Lipshitz continuous (i.e. continuous, differentiable and bounded gradient, transcendental and monotonically increasing) [5, 34]. In the case of the standard ESN, the widely used transition function is the hyperbolic tangent $\tanh(\cdot)$, and the readout expression (2) is a linear model.

As we already mentioned, a distinctive characteristic of the ESN model is that the reservoir weights (matrix $\mathbf{W}^{\text{h}}$) are frozen during the learning phase. Therefore, the transition function is fixed for all the instances; for this reason, (1) is sometimes referred to as static random projection. Another common word for expression (1) is *expansion function*, because the dimension of the reservoir must be much larger than the dimension of the input space [2]. The readout parameters ($\mathbf{W}^{\text{out}}$) can be learned using the regression (2) and applying a classical tool such as Tikhonov regularization [34].

The reservoir initialization may have an impact on the model's performance [25, 50]. In addition, there are several hyperparameters that are relevant in the robustness of the ESN [18]. The *reservoir size* defines the dimensionality of the reservoir projection. Large reservoirs may create meaningful feature subspaces and improve the separability of the input data. However,

to increase the complexity may provoke overfitting. The *input scaling factor*, as its name specifies, defines the domain of the input weight matrix. Therefore, its value may impact on the recurrences, and then modify the reservoir state trajectories. The *neuron activation functions* impact in the ability to memorize the input sequence, as well as in the linear separability of the input data in the feature space. The most commonly used function type is the hyperbolic tangent [2, 4]. Another hyperparameter is the *sparsity* of the reservoir weight matrix. The RC literature suggests using a sparse reservoir weight matrix, with about 15% to 30% of non-zero values [18, 50]. Sparsity doesn't significantly affect the model's accuracy [50]. However, it naturally has a strong impact on the computational costs: a sparse reservoir matrix enables fast reservoir state updates. The most common readout mapping is the linear regression, but other procedures have been used such as Support Vector Machines, single layer networks or PCA [4]. The pattern of connectivity is another important aspect at the moment of designing an initial reservoir. Reservoirs with specific cyclic jumps are analyzed in [21]. In particular, the impact of the degrees of the reservoir nodes (graph theory meaning) on the global model accuracy [29]. Multilayer readouts and hierarchical reservoirs have been developed during the last years [4]. This is a new promising area inside the RC family. Hierarchical reservoirs, sometimes referred to as deep reservoir architectures, are a kind of cascade of reservoir clusters with connections between them [15, 16, 51]. The simplicity and efficiency of the RC computational schema has made that several works in the community have addressed the problem of implementing these architectures in hardware [4, 52, 53].

3.2. Recurrence stability

Several studies analyze the stability conditions and asymptotic properties of the recurrences of expression (1) [18, 31, 34, 54]. A structural and fundamental property is the *Echo State Property* (ESP). The ESP guarantees that the reservoir state becomes asymptotically independent of the initial conditions [55]. Even if the initial trajectories are strongly random, the reservoir projections should be independent of them in the long term. Additionally, the network should satisfy a type of "functional" relationship where each input sequence is associated with a single output sequence in the long term. These two properties are established in the ESP [2].

The spectral radius and the singular values of the reservoir matrix have an important impact on the ESP [2, 31, 34]. Previous mentioned studies

and experimental works suggest building reservoirs that hold the ESP, and a simple way to do it consists of scaling the reservoir weights using the spectral radius of the reservoir matrix [2, 31]. Let us denote the spectral radius of the $\mathbf{W}^h$ matrix by $\rho(\mathbf{W}^h)$. The common procedure for defining the initial reservoir is as follows: (i) random initialization of the reservoir weights $\mathbf{W}_{(\text{rand})}^h$, (ii) computation of its spectral radius $\rho(\mathbf{W}_{(\text{rand})}^h)$, (iii) scaling operation $\mathbf{W}^h := \alpha \mathbf{W}_{(\text{rand})}^h / \rho(\mathbf{W}^h)$, where α is a parameter in $[0, 1]$. Using this initialization, the final matrix will have a spectral radius equal to the predefined α value. Actually, pathological situations were analyzed by Jaeger, and it is enough to properly scale the reservoir weights to almost always satisfy the ESP [2]. Moreover, the property is violated in autonomous networks without external entries if $\rho(\mathbf{W}^h) > 1$ using a convex activation function [2]. In this particular case, the reservoir state may present oscillations or a chaotic behavior [2, 34]. Also, the property may be unsatisfied when the spectral radius is less than 1, but it corresponds to unusual and strange topologies (not used in practice) [50]. It has been found that optimal computational performance of the reservoir units operates in a regime that lies between stable and chaotic behavior [7, 14, 18].

3.3. Modern ESN variations

In recent years, several variations of the canonical ESN have emerged. We integrated some of them in our experimental results.

Feedback connections [28]. The recurrent state is computed by

$$\mathbf{x}(t) = f(\mathbf{W}^{\text{in}}\mathbf{u}(t) + \mathbf{W}^h\mathbf{x}(t-1) + \mathbf{W}^{\text{fb}}\mathbf{y}(t)), \quad (3)$$

where $\mathbf{W}^{\text{fb}}$ is a weight $p \times o$ matrix with the feedback connections going from the output neurons to the hidden structure. The feedback can also be integrated with an arbitrary delay. When the model has feedbacks, the reservoir state is driven by both the input signal and the model output, and this increases the computational power of the neural nets [56]. Nevertheless, stability issues may arise with feedback connections [50].

How to train feedback weights? In the case of models with feedback, a common strategy called *teacher-forcing* is usually applied for training the readout weights [28, 57, 58]. The schema is as follows. During the training phase, the feedback weights are teacher-forced, i.e. instead of feeding back the reservoir with the predicted value $\mathbf{y}(t)$, the real target is used as the

feedback value [57]. In exploitation mode, a free-run prediction is realized, where the prediction $\mathbf{y}(t)$ is fed back to the reservoir.

Additional noise [29]. The reservoir state with injected noise has the form:

$$\mathbf{x}(t) = f(\mathbf{W}^{\text{in}}\mathbf{u}(t) + \mathbf{W}^{\text{h}}\mathbf{x}(t-1) + \nu(t)), \quad (4)$$

where a tiny noise $\nu(t)$ is incorporated during the training procedures. To add noise is a common regularization strategy for increasing the generalization capability of the model. The noise is injected only during the training phase, in exploration mode the noise parameter $\nu(t)$ is set to zero.

Leaky integrator ESN. The reservoir state variation is smoothed as follows:

$$\mathbf{x}(t) = (1 - \gamma)\mathbf{x}(t-1) + \gamma\phi(\mathbf{W}^{\text{in}}\mathbf{u}(t) + \mathbf{W}^{\text{h}}\mathbf{x}(t-1)), \quad (5)$$

where $\gamma \in (0, 1]$ is the *leaking rate* parameter [2]. It controls the sensitivity over the entry values, and has an impact on the “speed” of changes in the trajectories [50].

4. EvoESN: Evolutionary Echo State Networks

4.1. Discrete Cosine Transforms

The Discrete Cosine Transform (DCT) satisfies properties that make this mapping suitable for encoding large matrices [22, 25, 59]. We use one of the most common versions, the normalized DCT of Type II): given an arbitrary sequence of H reals $s = (s_0, s_1, \dots, s_{H-1})$, its DCT is $\text{DCT}(s) = S = (S_0, S_1, \dots, S_{H-1})$, where $S(0) = \sqrt{1/H} \sum_{h=0}^{H-1} s_h$ and

$$S_\ell = \sqrt{\frac{2}{H}} \sum_{h=0}^{H-1} s_h \cos\left(\frac{\pi\ell(2h+1)}{2J}\right),$$

for $1 \leq \ell \leq H-1$. This transformation has an inverse, denoted IDCT here, that is, with previous notation, $\text{IDCT}(S) = s$, where

$$s_\ell = \frac{S_0}{\sqrt{H}} + \sqrt{\frac{2}{H}} \sum_{h=1}^{H-1} S_h \cos\left(\frac{\pi h(2\ell+1)}{2H}\right).$$

Our proposed approach EvoESN relies on the bijectivity and the energy compaction properties of the DCT. The first point is an immediate consequence of the linearity of the DCT and the IDCT. So, they are actually C^∞

functions from $\mathbb{R}^H$ onto $\mathbb{R}^H$. The second property means, informally, that we can choose some $K < H$ (typically, $K \ll H$) such that if S' is built by keeping the first K elements of $S = DCT(s)$ only (called the *low frequency* terms), replacing the rest by 0s, then $IDCT(S')$ is pretty close to s .

4.2. Evolutionary algorithms on modern ESN models

The survey article [2] covers the first decade of the RC area, including some initial literature regarding evolutionary methods for optimizing reservoirs. More recently, evolutionary algorithms have been used as hyperparameter optimization in the RC area. Both PSO and GAs have been used in [9, 11, 60, 61, 62]. Authors employed EA for tuning global parameters such as the number of reservoir units and the scaling factors of connectivity properties (density and spectral radius of the reservoir weights matrix).

GAs for finding the global parameters of an extended ESN for modeling medical signal prognosis were studied in [10]. A pre-training algorithm based on swarm optimization was developed in [12]. The authors applied PSO for adjusting a subset of the reservoir weights. Instead of searching the global parameters, the proposed technique works directly on a subset of reservoir weights. In [13, 14] the idea of growing RC topologies using Neat and HyperNEAT was investigated. In [14] the evolution of the reservoir topology was made considering a fitness function that combines both the memory capabilities and the model's accuracy. Evolutionary techniques were also applied for optimizing parameters in hierarchical reservoirs, for instance, for estimating the weight connections between two consecutive reservoirs in the hierarchy [15, 16]. A variation of GA named microbial GA was used in [16]. In addition, the weight connections among the reservoirs are set in order to make a global contraction mapping (a kind of similar restriction than the ESP in classic ESN). Recently, PSO was introduced as a hybrid approach of swarm optimization and local search in hierarchical ESNs [63].

4.3. Indirect encoding in the Fourier space

In [1, 24], an indirect encoding schema for NNs was introduced, where weight connections were represented, in a compressed form, in the frequency domain. Authors apply EAs where the individuals (chromosomes) are represented by the type-II DCT of the weight matrix. That is, the search space in the evolutionary phase is composed of vectors of Fourier-like series coefficients. In parallel, in the original application the authors define a coevolutionary method, called CoSyNE, for searching in the space of the DCT

coefficients. Evolino networks, based on an LSTM network, is another example where the network weights are represented in another space and the EAs approach is adopted for solving the optimization side [1]. This approach has been successfully applied over rather small fully connected RNNs on three benchmark problems of the area of control: pole balancing, ball throwing and octopus arm [24]. The authors have also applied this encoding procedure in other problems related to computer vision and Reinforcement Learning [22]. The encoding was recently re-visited in the NN domain [25, 59]. In [59], the authors performed a DCT encoding over a LSTM network, and the model is applied to solve language modelling problems. The EvoESN, introduced in [25], presents a fine-tuning procedure for optimizing the reservoir weights based on the indirect mapping DCT and on GAs [25].

4.4. Neuroevolutionary framework

In this section, we describe the EvoESN computational model, and describe its main characteristics and benefits.

Notation. We first choose the number M of (a priori) non-null weights in the reservoir. Reservoirs are commonly designed to be sparse, then we assume that $M \ll p$. We also choose which weights in the reservoir will be frozen to 0, by randomly choosing the remaining M positions among the total p^2 possible ones. We define two auxiliary vectors γ and ν representing positions in $\mathbf{W}^h$ (i.e. (row, column) pairs) and corresponding weights' values, respectively. If $\gamma_k = (i, j)$, we mean that the k th position in γ refers to the weight in row i and column j of $\mathbf{W}^h$. The vector ν (with dimension M) contains those weights, that is, $\nu_k = \mathbf{W}_{\gamma_k}^h = \mathbf{W}_{i,j}^h$. During all the procedures, the positions (a kind of hash map) will remain fixed; only the corresponding weights will change following the evolutionary search. That is, γ remains fixed and ν evolves during the execution of the proposed method. We denote by C the number of DCT coefficients to be used for representing ν . The integer C must satisfy $C \leq M$. Besides, we will denote by α the vector with the first C coefficients $\alpha_1, \dots, \alpha_C$ of the DCT of ν .

Coefficient interpretation. As in any application of Fourier-like transformations, the frequency used in α_i is lower than that in α_j when $i < j$, and lower frequency coefficients store more information than higher frequency ones. We say that the coefficient α_i is more significant than α_j when $i < j$. So, the higher C , the better in principle the accuracy of the application, but of course, the higher the computational cost.

Evolutionary search. It consists of moving the vector ν to the frequency domain using the DCT transform, to change it running an evolutionary procedure, to go back to weights through the inverse DCT, to evaluate the introduced changes by training the ESN with the just obtained reservoir, and to repeat this process until some convergence condition is satisfied. As we will see, we actually never need to perform a DCT, only its inverse is required in the algorithm.

The first step is to choose the initial values of the C coefficients. We discuss this initialization procedure in Subsection 4.6. Denote now by $\phi(S)$ the inverse DCT transformation of sequence S (IDCT). If we write $s = \Phi(S)$, both sequences S and s have the same size. To refer to the evolutionary process used in [25], a genetic one, we will also call *chromosome* the DCT of vector ν (that is, vector α), and ν itself will be called *phenotype*. In case $C < M$ (the usual one), we need to extend the chromosome α with $M - C$ zeros (the operation is called *padding*) to obtain a vector $\phi(\alpha)$ of dimension M . The padding operation is a standard method used to increase the values in the frequency space. We also call *extended chromosome* the vector α when $C < M$. Once we have possibly extended the chromosome α and computed $\nu = \Phi(\alpha)$, the ESN is trained as in the classic procedure. If the test step doesn't validate the training, we execute an evolutionary search in the frequency space, in order to obtain a new chromosome α , leading to a new phenotype $\nu = \Phi(\alpha)$. The new ESN is evaluated. The whole procedure is repeated until some convergence condition is satisfied. Figure 1 visualizes the global procedure. The proposed framework has the following main operations:

- (i) Initialization of the chromosomes (sometimes called particles, depending on the terminology used by the EA).
- (ii) Padding operation: it is the chromosome extension, in which zero values are concatenated until dimension M is reached.
- (iii) Indirect encoding: IDCT application.
- (iv) Genotype-phenotype mapping: to assign the elements of the phenotype to their positions in the reservoir.
- (v) Fitness computation: for each chromosome, training of the readout structure and ESN evaluation over testing data.

- (vi) Evolutionary search: to define a protocol for changing from one generation to the next one.

Pseudocode description. We describe the proposed computational model in two pseudocodes: the evolutionary search is presented in Algorithm 1 and the fitness function computation for each candidate (particle/chromosome) is presented in Algorithm 2. Notice, that the particle extended chromosome is first transformed into the original space of weights (through $\boldsymbol{\nu} = \phi(\boldsymbol{\alpha})$), and then, before training, it is possible to re-scale the reservoir weights (in order to control the ESP). This operation of re-scaling (line 8) is optional. In the experimental section, we present results with scaling and without scaling the reservoir weights.

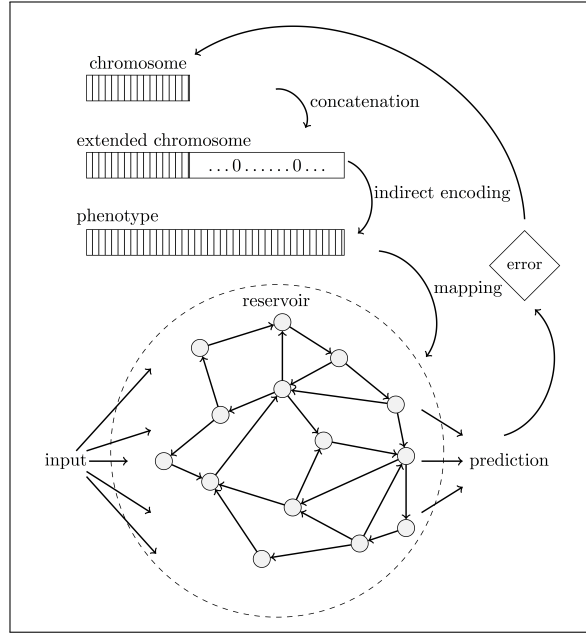


Figure 1: Visualization of the genotype-phenotype mapping of EvoESN. We see the chromosome (the vector of the DCT transform of the non-null weights in the reservoir), extended with zeros if necessary, then sent to the weight space as a new phenotype through the inverse DCT (we also call this *indirect encoding*), used by the ESN to make, after training, a prediction. The error is considered in the evolutionary process for computing the fitness value of each chromosome.

4.5. Evolutionary search using PSO

In the introductory work about EvoESN presented in [25], the evolutionary search was performed using GAs. Here, we also incorporate the results of swarm optimization [26], because of the improvement they bring to the project. The PSO evolution is based on a simulation of swarming behavior of birds within a flock, as well as other groups of social animals [64]. At each iteration, the particles in the swarm exchange information about their positions. Each particle is represented by its position and velocity, and it can be interpreted as a multidimensional moving point with an associated velocity. The common notation in the literature for both variables are $\mathbf{x}$ and $\mathbf{v}$. Though we have already denoted the reservoir state with $\mathbf{x}$, in this brief PSO description we will anyway use the notation $\mathbf{x}$ for the particle position. In addition, each particle i remembers its best reached position $\mathbf{l}_i$ (in terms of fitness function), and it knows the best multidimensional position discovered so far by the whole swarm, $\mathbf{g}$. The position represents the actual candidate problem solution, and the velocity determines regions in the searching space to be explored. Here, we use indistinctly the terms particle and chromosome. The points $\mathbf{l}_i$ and $\mathbf{g}$ are sometimes referred to as the best local position of i and the best global position of the swarm. In each iteration t , the velocity of particle i is updated according to [27]:

$$\mathbf{v}_i^{(t+1)} = \iota \mathbf{v}_i^{(t)} + c_1 r_1^{(t)} (\mathbf{l}_i^{(t)} - \mathbf{x}_i^{(t)}) + c_2 r_2^{(t)} (\mathbf{g}^{(t)} - \mathbf{x}_i^{(t)}), \quad (6)$$

where ι is called inertia parameter, c_1 and c_2 are positive acceleration parameters and r_1 and r_2 are vectors of random values sampled from the uniform distribution. The inertia controls the direction of future explorations. The parameters c_1 and c_2 are called the *cognitive* and the *social* parameters, respectively. They control the trade-off between exploration and exploitation. Vector $\mathbf{l}_i^{(t)}$ represents the best position known by the particle i until iteration t , and vector $\mathbf{g}^{(t)}$ is the best position visited by the whole swarm also until t . The position of particle i is updated by [27]

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)}. \quad (7)$$

Due to its simplicity and good performances, there are many works regarding swarm optimization both in theory and in applications [27, 61, 65]. In the experimental section, we follow a suggestion of [27] for tuning the basic PSO parameters. We empirically analyze the inertia values using a grid search

in $[0.5, 1]$ with the additional condition that $1 > \iota > 0.5(c_1 + c_2) - 1 \geq 0$. The trajectory of a particle converges if the previous condition is held [27]. In addition, we explore the searching space updating at each iteration t the parameters c_1 and c_2 in order to avoid stagnation as follows [27]:

$$c_1^{(t)} = (c_{1,min} - c_{1,max}) \frac{t}{T} + c_{1,max} \quad \text{and} \quad c_2^{(t)} = (c_{2,max} - c_{2,min}) \frac{t}{T} + c_{2,min}, \quad (8)$$

where $c_{1,min}$, $c_{1,max}$, $c_{2,min}$ and $c_{2,max}$ are pre-selected constants (minimum and maximum values of c_1 and c_2), and T is the maximum number of iterations. In our problem, the exploration space is composed of the vectors of coefficients, living in the frequency domain.

4.6. How to properly initialize the chromosomes?

The IDCT operation transforms points from the frequency domain into the weight space. The weights should belong to an appropriate domain in order to avoid neuron saturation. Large weight values will provoke the neuron saturation phenomenon, where the reservoir state will predominantly be close to the asymptotic ends of the activation function (hyperbolic tangent). On the other side, excessively small weight values may slow down the training progress and incorporate some numerical issues.

In general, the initialization of the particles has a crucial impact on the global approach. Figure 2 shows the impact of the particle initialization. As an example, the particles have a dimension of 100, and there are 100000 non-zero values of reservoir weights. The three figures in the first column show the particle values. The swarm is initialized with different uniform distributions ($\mathcal{U}[0, 1]$, $\mathcal{U}[-1, 1]$, $\mathcal{U}[-4, 4]$). The three graphics, in the second column, show the values of the obtained weights after performing the padding operation and the IDCT function. It is common to initialize the reservoir weights using a symmetric random initialization with distributions $\mathcal{U}[-0.2, 0.2]$ or $\mathcal{U}[-0.5, 0.5]$ [28, 50]. Therefore, in our experiments, we increase the initial diversity of the swarm with particle initialization using different uniform distributions. We initialized particles with elements in the range $[-a, a]$ with $a = 4, 5$ and 6 .

5. Experimental analysis

We evaluate the EvoESN tool on three selected dynamical systems, and we compare with the standard ESN and its popular modern variations discussed in Section 3.3. We used the Mackey-Glass system (MGS), the Lorenz

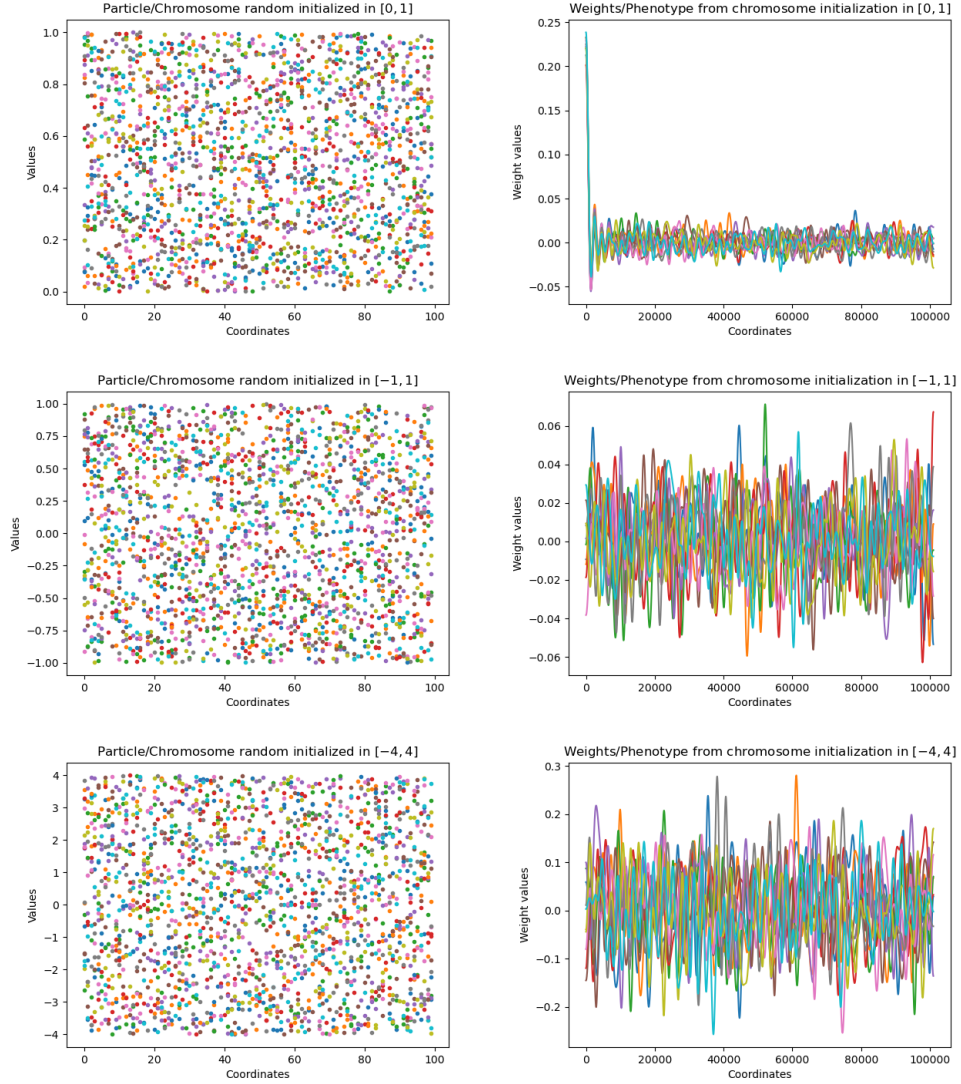


Figure 2: Impact of the initialization of the chromosomes on the weight space. The IDCT encoding projects the extended chromosome in the weight space. However, the domain of the weight space is relevant for avoiding neuron saturation. The figure shows how different uniform distributions obtain pretty different domains of the weight space.

Algorithm 1: Evolutionary search using PSO for reservoir weights in the Fourier space.

Inputs : Fitness function, learning data, PSO parameters, ESN parameters, network model.

Outputs: Best global particle.

// Swarm initialization using Section 4.6.

- 1 Initialization of $\{\alpha^{(1)}, \dots, \alpha^{(I)}\}$;
- // Evaluation of each particle.
- 2 Compute fitness for each $\alpha^{(i)}$ using Algorithm 2;
- // Apply evolutionary search.
- 3 **while** (Convergence criterion is not satisfied) **do**
- 4 Update particle velocity for all i using expression (6);
- 5 Update particle position for all i using expression (7);
- 6 Update inertia and acceleration parameters (ι, c_1, c_2) using expressions (8);
- 7 Update local best for all i ;
- 8 Update global best;
- 9 Compute fitness for each $\alpha^{(i)}$ using Algorithm 2;
- 10 Return best global particle;

Algorithm 2: Computation of the fitness value.

Inputs : Input particle, learning data, ESN parameters, network model.

Outputs: Fitness evaluation.

- 1 **if** ($C < M$) **then**
- // Chromosome/particle extension.
- 2 Padding operation for α ;
- // Frequency to weight domain.
- 3 $\nu = \text{IDCT}(\alpha)$;
- // Filling reservoir.
- 4 Assign ν values to $\mathbf{W}^h$ according to positions in γ ;
- // Optional operation.
- 5 Reservoir scaling to satisfy ESP;
- // ESN training.
- 6 Train readout weights;
- 7 Return fitness error for the evaluated particle;

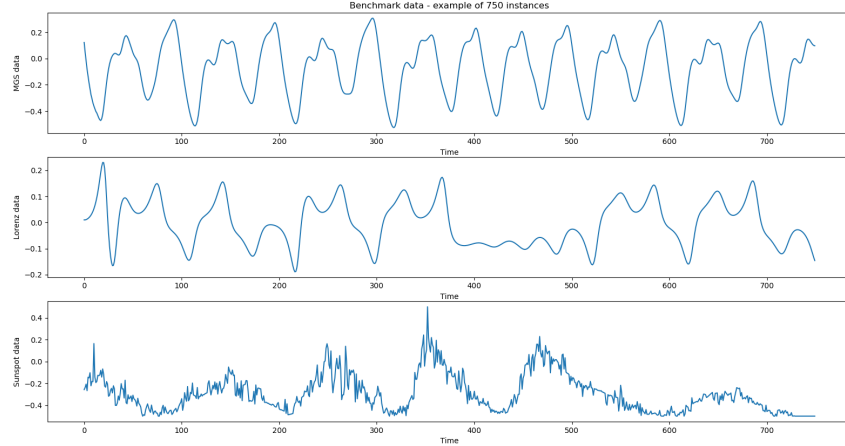


Figure 3: Visualization of 750 instances of the three investigated problems.

system and the Monthly Sunspot index data, since those benchmark problems cover different types of chaotic time-series and have been largely studied in the RC community [1, 7, 28, 29, 32]. Lorenz and MGS problems have been tested in almost every nonlinear system modeling [28]. The Monthly Sunspot data has measurements of the magnetic field in the Sun’s outer regions. The series is publicly available at [33]. Sunspot data has also been analyzed using RC models in at least [15, 19, 29, 66]. Figure 3 shows 750 instances of the three studied problems, where it is possible to see the diversity in the behavior of the three chaotic time series. For experimental reproducibility, benchmark datasets and source code are available in: <https://github.com/sebabaster/EvoESN>.

5.1. Benchmark problems

Theoretical data: Mackey-Glass and Lorenz systems. The standard ESN model acquired a wide recognition for their obtained success in forecasting the MGS data [28]. Actually, the record-breaking achieved accuracy in modeling MGS instances was obtained with one of the ESN variations presented in Section 3.3, where the recurrent state operates with an injected noise and feedback connections. The MGS model is a type of nonlinear auto-regressive (NAR) dynamical system used for analyzing bifurcations in

physiological applications. It is defined by the differential equation

$$\frac{\partial y}{\partial t} = \frac{\alpha y(t - \tau)}{1 + y(t - \tau)^\beta} - \gamma y(t),$$

with parameters α, β, γ and τ . Despite its simplicity, this equation can generate periodic and complex chaotic dynamics. We assigned to the parameters the following values, that are the most commonly used in the literature: $\alpha = 0.2$, $\beta = 10$, $\gamma = 0.1$ and $\tau = 17$. Chaotic attractors appear when the parameter τ (so-called delay) is larger than 16.8 [28]. The data preprocessing and experimental setup was designed following the setting realized in [28, 25]. An initial time window with 1000 samples was used for cleaning up the initial transient phase. The training phase was made with 3000 points. The testing phase has 2084 samples, in which we used the first 2000 time steps in a teacher-forced mode. From this last time step, we run the network in exploitation mode.

The well-known Lorenz system is defined by the differential equations

$$\frac{\partial x}{\partial t} = \sigma(y - x), \quad \frac{\partial y}{\partial t} = rx - y - xz, \quad \frac{\partial z}{\partial t} = xy - bz.$$

This system was initially developed as an attempt to model the atmosphere convection. Lorenz's equations have received a lot of attention because the system is presented also in other physical phenomena such as the dynamics of electro-rotation, thermohaline ocean flow circulation, Malkus waterwheel and dynamics of whether transition [67]. The data was generated using the original parameters in the equations: $r = 28$, $b = 8/3$, $\sigma = 10$. The generated data in the x -coordinate trajectory was rescaled by a factor of 0.01, following [28]. Then, the obtained series is used for the learning process. For the learning procedures, we replicated the experimental setting presented in [28]. The first 1000 samples were used for removing the transient effects, that is, for eliminating the impact of the initial conditions. The training sequence has a length of 6000 samples, and the testing sequence has 1600 points. The first 1000 samples were used in teacher-forced mode, and the last 600 points in exploitation mode.

Real world data: Monthly sunspot series. It has been observed that the Sun has occasionally sunspots, which became a distinctive feature of the solar cycle. This series is used for obtaining a better knowledge of the solar cycles, and insights regarding the evolution of the total number of spotless days [33].

We used a sequence with 3276 data points from the period Jan. 1749 to Dec. 2021 [33]. The experimental setting was replicated from the experiments analyzed in [25, 29]. We standardized the variance and normalized the data in $[-0.5, 0.5]$, in order to obtain a better control of the recurrent stability and to avoid neurons’ saturation. We used an initial window with 100 samples for washout the initial transient. For the learning procedure, we used 1600 instances for training and 500 instances for testing evaluation.

5.2. Results

Evaluation metrics. For the model evaluation, we replicated the experimental protocol conducted in [25, 28]. The general procedure is as follows. We ran the trained model over the testing dataset, and we discarded a washout with initial transients. Then, we computed the forecasting error using H time steps ahead. The estimation of the predictive error was made performing several metrics. We used normalized-root-mean-squared error (NRMSE) for the MGS and Lorenz problems, that was also used in [1, 28, 32]. In the case of the sunspot series problem, we used the normalized-mean-squared error (NMSE), since it was employed in [29]. In addition, for the sensitive analysis of the parameters, we used the root-mean-square error (RMSE) computed over a time windows with H samples. For a better visualization, we used a 10-logarithmic scale in the error curves.

In the RC area, it is common to see the evaluation of the learning procedure using two criteria, often denoted as $\text{NMRSE}(H)$ and NMRSE_H . $\text{NMRSE}(H)$ denotes the NMRSE computed in the testing phase using the residuals over *the whole horizon* H . The NMRSE_H is computed in the testing phase after H autonomous steps, and the error is computed only using the residual at the H time step [1, 28]. The NRMSE_{84} metric has become the standard error function for evaluating the MGS prediction [1].

Hyperparameter selection. We conducted a grid search strategy to define the main global parameters. We also follow suggestions from the literature at the moment of defining parameters, such as network size, input scaling, added noise, non-linearity type of projections, and feedback connections [28]. A grid search in the range $[0.1 : 0.1 : 1.4]$ was performed for the spectral radius of the reservoir matrix. Even though the ESP is not guaranteed, we evaluated matrices with spectral radius larger than 1, similarly than [11, 20, 32]. Actually, in our experiments, we figured out that some reservoirs with large spectral values also reach good accuracy for a specific window forecast. For

solving the Sunspot task, we adopted both grid search and the experimental setting employed in [29]. Following [29], the objective in the Sunspot problem is to predict one single time-step ahead [29]. In other words, the network doesn't perform in exploitation mode over large testing time windows. In the case of the Sunspot task, we studied only networks without feedback connections. We did a grid search in $[0.1 : 0.1 : 0.5]$ for the sparsity parameter. The leaky rate was set in the range $[0.01 : 0.05 : 0.5]$. The regularization parameter used for computing the readout weights was evaluated with values 10^{-i} , with $i \in \{3, 4, 5, 6, 7, 8\}$. The input scaling factor was evaluated in the set $\{0.01, 0.02, 0.03, 0.05, 0.07, 0.09, 0.1, 0.15, 0.2, 0.3, 0.5\}$, and it was selected the 0.1 value. The pattern of connectivity of the reservoir was randomly initialized using uniform distributions. In addition, we filled the upper diagonal of the reservoir matrix with random values, and also added a random value in the last row and first column. This step is commonly made to guarantee at least one cycle in the reservoir. The range of the uniform distribution was specific for each problem, following the suggestions discussed in [28]. The readout was computed using a ridge regression with a regularization factor of 10^{-9} (MGS prediction task), 10^{-6} (Lorenz prediction task) [28], and 10^{-5} (Sunspot prediction task).

The PSO parameters have been tuned following the suggestions presented in [27]. Figure 4 shows an empirical analysis of the impact of the inertia values on the evolution. A grid search in the range $[0.5 : 0.1 : 1]$ with the additional convergence condition $1 > \iota > 0.5(c_1 + c_2) - 1 \geq 0$ was performed [27]. The c_1 and c_2 values were initialized in the interval $[0.5, 2.5]$ [27]. Table 1 presents a statistical analysis for different values of the inertia parameter. We performed a total of 10 experiments with different inertia values over the MGS data, then we computed 95% t-student confidence intervals that are shown in Table 1. Consequently, we selected an inertia value equal to 0.9 that we used in our further experiments. Another hyperparameter is the initialization procedure for the coefficients, which defines the initial particles. We partially investigated its impact. Some results are presented in Figure 2. We decided to create swarms with particles initialized in a different range of values. The swarm has particles randomly initialized with uniform distribution in $[-a, a]$ with $a \in \{4, 5, 6\}$.

Results analysis. Figure 5 has two graphics showing the error in logarithmic scale, for MGS and Lorenz tasks. The curves for different values of C show the evolution of the average of RMSE across the 10 independent ex-

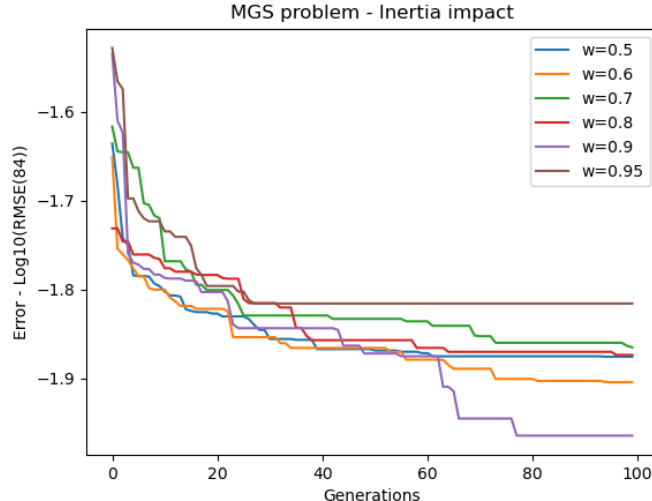


Figure 4: Error evolution according to the inertia parameter. Curves present the results of swarm experiments. It is shown the error average obtained by 5 different initial populations. The particle size is 75 and the population size was 10. Metric error was the RMSE over a testing window with 84 points.

periments. In these experiments, the particles were initialized with values in $\mathcal{U}[-a, a]$ with $a = \{4, 5, 6\}$, and the spectral radius of the reservoir matrix wasn't scaled. In other words, the swarm has an initial diversity with respect to the reservoir matrix spectrum's. The graphs in Figures 5 also have a constant curve labeled ESN+ with 95% confidence interval margins (computed over an average of 30 experiments). It shows the error obtained with the "optimal" ESN architecture (according to the results in [25, 28] using our own implementation). Remarkably, a few swarm iterations are enough to overcome the results achieved with baseline modern ESN. Note that ESN+ is a complex model with feedback connections, injected noise, regularization parameter, etc. We explicitly showed the ESN+ performance as a kind of threshold obtained by the state of the art. Observe that the accuracy becomes roughly independent of C . These results aren't consistent with previous ones where the evolution was made using GAs [25]. In the case of applying GA, larger chromosomes obtained a better accuracy. In the case of applying PSO, this seems to behave differently. Maybe, it is related to the evolutionary search characteristics of the PSO algorithm, where the small-size particles explore more efficiently the searching space. A further

Table 1: Sensitive analysis of the inertia parameter of the PSO algorithm with 75 particles. The 95% confidence interval was computed using the results of 10 explorations with 100 generations each of them. The error is the RMSE over a window of 84 time steps.

Inertia	Mean	CI lower bound	CI upper bound
0.5	0.0133	0.0116	0.0151
0.6	0.0125	0.0109	0.014
0.7	0.0136	0.0112	0.0161
0.8	0.0134	0.012	0.0147
0.9	0.0109	0.0072	0.0145
0.95	0.0153	0.0146	0.0159

analysis regarding the relationship between the C value, population size and exploration-exploitation characteristics of the evolutionary search can eventually also be studied in the future. Figure 6 presents results obtained with the MGS data. It illustrates the impact of the initialization of the coefficients when the reservoir matrix is scaled for controlling the initial spectral radius. In addition, the graphic in the right shows the evolution of the spectral radius through the generations. Occasionally, the best swarm solution represents reservoirs with spectral radius larger than 1. Figure 7 has four graphics; it visualizes the advantages of starting the swarm with particles that represent reservoirs with controlled spectral radius. The graphics were done with the Lorenz data, but the same behavior has been presented in other analyzed tasks as well. The spectral radius of the reservoir is a central parameter of the model. To initialize particles representing reservoirs with spectral radius in an appropriate range is better than a total random initialization, the error decreases much faster during the evolution. Also, it is visible that some best particles in the swarm represent reservoirs with spectral radius larger than 1. Therefore, the swarm is also able to explore other regions. In other words, reservoirs with spectral radius larger than 1, are also evaluated during the evolutionary search.

Comparisons with related works. Table 2 summarizes the results obtained for the MGS problem. In [28] was introduced a learning approach for correcting the bias introduced by the teacher-forcing schema. In addition to the readout training, the authors apply a training for bias correction. As far

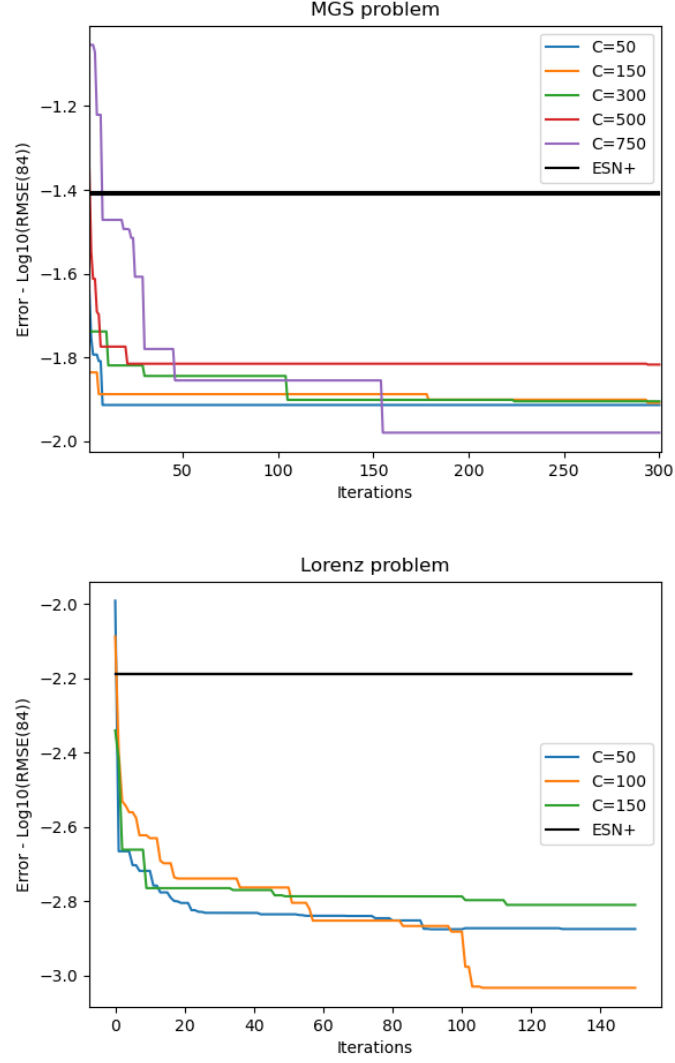


Figure 5: Impact of the number of coefficients on the accuracy. The results presented with EvoESN were computed without spectral radius control over the reservoirs matrices. The constant curve (ESN+) shows the reached accuracy by the ESN with noise and feedback connections designed for MGS and Lorenz problems in [28].

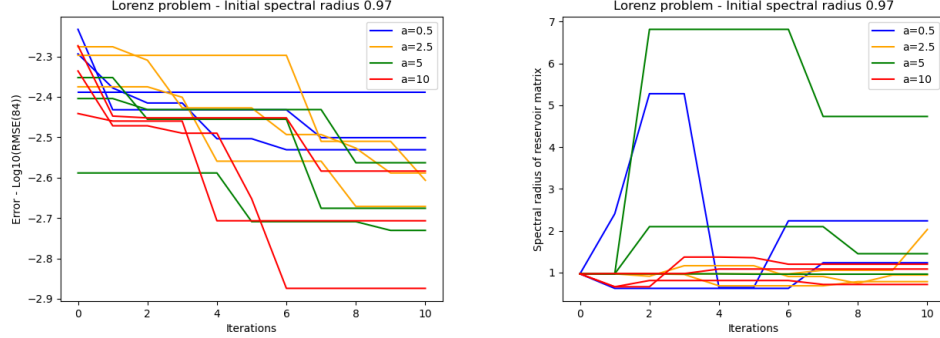


Figure 6: Impact of the initialization of the coefficients. The initial spectral radius of the reservoir was 0.97. The coefficients were randomly initialized using the uniform distribution in $[-a, a]$. The graphic on the left shows the error evolution in the first 10 iterations. On the right side, we show the evolution of the spectral radius of the best global particle.

as we know, Jaeger *et al.* obtained the best performance for the MSG task using this specific algorithm, that they called *refined learning method*. We evaluated the model using linear and hyperbolic tangent readout. Table 2 also shows the results obtained by EvoESN with GA, it presents the average of errors after 50 trials obtained using GA with chromosomes with a size of 500, a population of 20, and 70 and 300 generations. More information about the GA implementation in the EvoESN model is available in [25]. Note that EvoESN slightly overcomes the SOTA accuracy reported in [28]. However, even though the variance is very low, we cannot say that the difference is significant with the error reported in [28] using the additional learning schema (ESN with FB (II), see Table 2). However, it is relevant to note that a few PSO iterations are enough to significantly decrease the error, as it was shown in Figure 5.

Table 3 shows results obtained for the Lorenz prediction task. The EvoESN with PSO has feedback connections and a non-linear readout. The initial spectral radius is controlled. The errors are averages computed over 5 independent experiments. The NRMSE₈₄ error using GA has slighter lower error than the result obtained by the refined learning schema [28]. PSO with a smaller number of coefficients (particle size) and in a half number of generations reach equal order of magnitude than GA and the SOTA result [28]. Table 4 presents the test errors for the Sunspot series problem. EvoESN (with GA and PSO) outperforms the results reported in [29]. Furthermore,

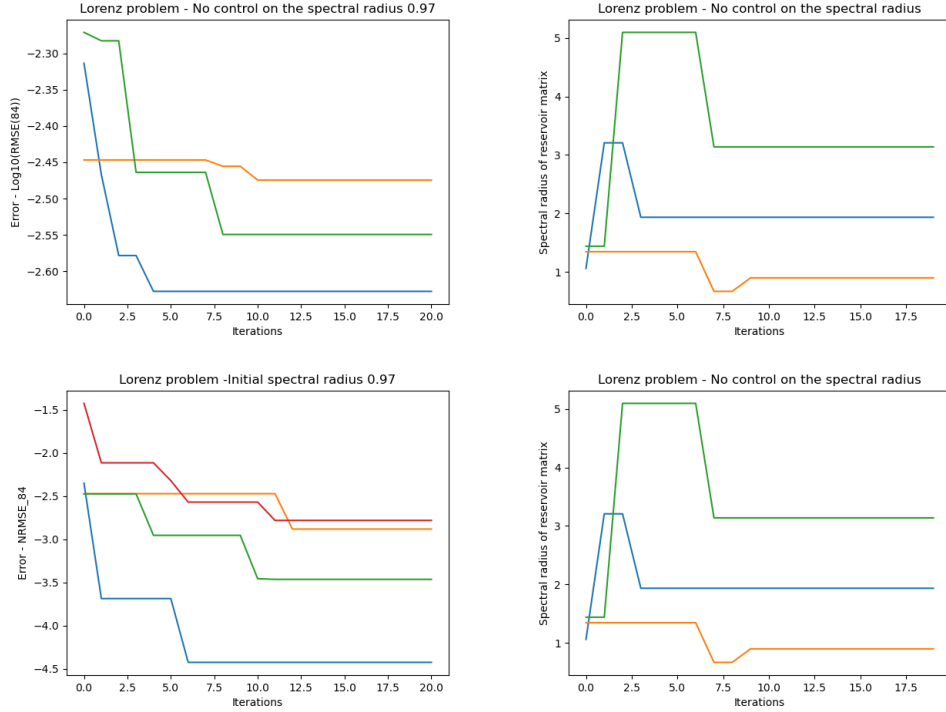


Figure 7: Impact of controlling the spectral radius of the reservoir. The coefficients were randomly initialized using a uniform distribution in $[-5, 5]$. The two figures above show results with reservoir without control of the spectral radius. The two figures below show results, with initial reservoirs with controlled spectral radius (fixed to 0.97). The graphics on the left shows the error evolution in the first 20 iterations. On the right side, we show the evolution of the spectral radius of the best global particle. The particle size is 50 and the population size is 10.

it also outperforms the results obtained by ESN with leaky integrator and ESN with feedback connections. We defined a randomly selected reservoir topology, and we ran 50 independent random weights initialization. Independently of the selected leaky rate, the obtained error variance has been small ($\approx 1 \times 10^{-6}$). For this problem (with the evaluated number of iterations), it is clear how PSO achieves a better accuracy than GA. Figure 8 presents the evolution of the error in logarithmic scale, the EvoESN with PSO is able to decrease the error very fast, only few generations are needed.

Algorithmic complexity. EvoESN has a higher computational cost than the standard ESNs. Note that at each generation, the EA performs the following operations: training of the readouts using a linear regression, computing the inverse DCT, and performing the evolutionary operations. Training the readouts needs matrix operations (matrix products, matrix times vectors, matrix inverses). Those operations also appear in the standard ESN. However, they are performed over each particle and per each generation in the EvoESN model. Other additional operation comes from the Fourier transform. A Fast Fourier transform (FFT) algorithm has a computational complexity of $O(M \log(M))$, where M is the number of Fourier coefficients. Evolutionary operations performed from one generation to the next one are less costly (matrix-vector operations). Resuming, the computational complexity of EvoESN lies on the FFT method and the selected EA. Nevertheless, we observed in the different experiments that a few generations of the PSO algorithm are enough for achieving much lower errors with respect to ESNs. According to our results, GA has a lower convergence speed than PSO, and also requires encoding the weights in larger vectors.

Computational aspects. The IDCT was chosen with the orthogonal norm using the `fftpack` Python package. PSO algorithm was designed according to the description presented in Section 4.5. The results of EvoESN using GA were conducted using the `DEAP` package [68]. Statistical tests have been done using the `Statsmodels` package [69]. The `Numpy` package was used for matrix operations such as spectral radius computation and matrix inverse [70].

6. Discussion

From previous theoretical analysis and the conducted experiments, we identify the following characteristics, advantages, limitations and future research directions.

Table 2: Summary of MGS results. Comparison between the proposed EvoESN optimized using GA and PSO with other ESN variations and results reported in the literature. ESN with FB (I) means ESN with feedbacks, and ESN with FB (II) means ESN with feedback and the refined learning method.

RC model	Characteristics	Error	Metric
H. Jaeger <i>et al</i> [28]	ESN-FB (II)	$\approx 10^{-4.2}$	NRMSE ₈₄
J. Schmidhuber <i>et al.</i> [1]	Evolino	$\approx 1.9 \times 10^{-3}$	NRMSE ₈₄
ESN	without FB, non-linear readout	0.1770	NRMSE (84H)
ESN	without FB, linear readout	0.29857	NRMSE (84H)
ESN	with FB (I), linear readout	0.19620	NRMSE (84H)
ESN	with FB (I), non-linear readout	0.19376	NRMSE (84H)
ESN	with FB (II), non-linear readout	0.07684	NRMSE (84H)
EvoESN (GA)	$C = 500$, 20 chromosomes, 70 generations	$\approx 4.485 \times 10^{-4}$	NRMSE (84H)
EvoESN (GA)	$C = 500$, 20 chromosomes, 300 generations	$\approx 1.075 \times 10^{-4}$	NRMSE ₈₄
EvoESN (PSO)	$C = 50$, 30 particles, 150 generations	$\approx 4.172 \times 10^{-4}$	NRMS ₈₄
EvoESN (PSO)	$C = 100$, 10 particles, 150 generations	$\approx 7.068 \times 10^{-4}$	NRMS ₈₄

Table 3: Summary of Lorenz results. Comparison between the proposed EvoESN optimized using GA and PSO with other ESN variations and results reported in the literature.

RC model	Characteristics	Error	Metric
H. Jaeger <i>et al</i> [28]	ESN-FB (I)	≈ -2.33	$\log_{10}$ NRMSE ₈₄
H. Jaeger <i>et al</i> [28]	ESN-FB (II)	≈ -4.27	$\log_{10}$ NRMSE ₈₄
EvoESN (GA)	$C = 50$, 300 generations	-3.4666	$\log_{10}$ NRMSE84
EvoESN (GA)	$C = 100$, 300 generations	-3.6014	$\log_{10}$ NRMSE84
EvoESN (GA)	$C = 150$, 300 generations	-4.1379	$\log_{10}$ NRMSE84
EvoESN (PSO)	$C = 50$, 30 particles, 150 generations	≈ -4.2852	$\log_{10}$ NRMSE ₈₄

Initialization scheme. We studied different initialization forms for the particles. According to the domains of the DCT mapping, we conclude that an appropriate choice is to use uniform distributions on $[-a, a]$ with, say, $a \in [4, 6]$, in order to avoid neuron saturation. Although, when we scale the initial reservoirs for satisfying the ESP, the type of random initialization for each particle does not significantly affect the results.

Dimensionality reduction. How to encode recurrent networks is still an open and rich research area. Here, we proposed an optimization procedure performed in a frequency space whose dimension is much smaller than the number of non-null reservoir weights. The “energy compaction” property

Table 4: Summary of Sunspot results. The EvoESN has feedback connection and non-linear readout. The errors are averages computed over 5 independent experiments.

RC model	Characteristics	Error	Metric
Rodan <i>et al.</i> [29]	ESN	≈ 0.1042	NMSE
Rodan <i>et al.</i> [29]	DLR, DLRB, SCR	≈ 0.1039	NMSE
ESN	without FB	0.085358	NMSE
LI-ESN	leaky rate 0.1	0.033120	NMSE
LI-ESN	leaky rate 0.7	0.031816	NMSE
EvoESN (GA)	$C = 100$, 150 generations	0.0292394	NMSE
EvoESN (GA)	$C = 200$, 150 generations	0.0293263	NMSE
EvoESN (GA)	$C = 300$, 150 generations	0.0309722	NMSE
EvoESN (PSO)	$C = 25$, 20 particles, 100 generations	0.001714	NMSE
EvoESN (PSO)	$C = 50$, 20 particles, 100 generations	0.001516	NMSE
EvoESN (PSO)	$C = 75$, 20 particles, 100 generations	0.000461	NMSE

of DCTs makes that few coefficients have enough information about the corresponding reservoir weights.

Particle/chromosome size. GA seems to be much more sensitive to the C value than PSO. Populations with larger chromosomes achieve lower errors, although the searching space exploration is much more costly. According to the results, we didn’t observe a similar behavior for the PSO algorithm. It seems that PSO is less sensitive to the particle size than GA. PSO can achieve similar results than GA with a smaller number of parameters. Even though the searching space of PSO has a lower dimensionality than the searching space of GA, PSO can achieve the same order of accuracy. Those results also suggest, for the near future, to realize a further investigation about the relationship among the dimensionality of the error landscape (given by C), the sampling strategies (affected by the population size, that is, the number of points used for covering the landscape) and the exploration strategy (which depends on the selected EA).

Control of the spectral radius of reservoir matrix. The results using PSO show the convenience of starting the evolutionary search with scaled reservoirs (matrices with appropriate spectral radius). A natural strategy can be to proceed to the selection of the spectral radius of the reservoir matrix following recommendations from the literature [2, 50], then to apply the PSO algorithm with particles representing reservoirs with the selected spectral

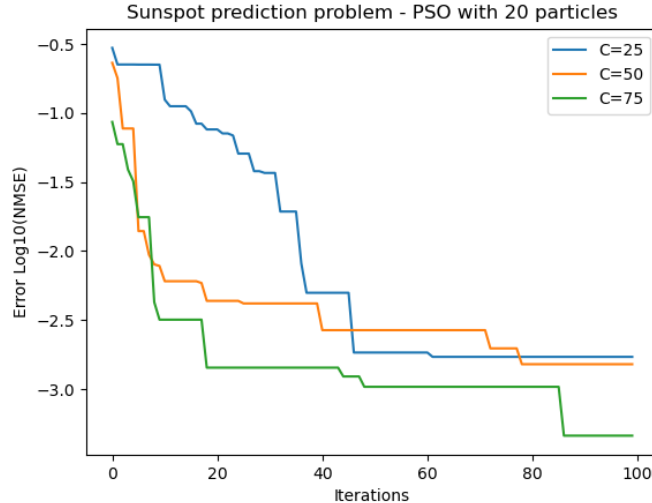


Figure 8: Error evolution for the Sunspot problem when it was used PSO with 100 particles. The error is presented in logarithmic scale for a better visualization.

radius. In our experiments for PSO, we did not re-scale the reservoir at each iteration. It was only computed for each particle in the initial population. Computing the spectral radius at each iteration will considerably increase the computational cost.

Initial diversity. We increased the initial diversity of the swarm by considering several values of $a \in [4, 6]$. It can also be interesting to explore, in the future, the impact of incorporating other type of diversity on the swarm. By diversity, we refer to the spectral radius of the reservoir that is a crucial parameter controlling the stability.

Decoupling between readout training and evolutionary fitness function. Note that the fitness function used in the evolutionary search can be different from the one used for training the readout weights. This opens several interesting possibilities, since the evolutionary optimization can be done over other properties of the reservoir, for instance, over stability control.

Searching strategy. Observe that the search for good reservoirs can include modifying the value of C . It is possible to start with a low value (low dimensional chromosomes) and then successively increase it for adding higher frequencies (the computational power of the reservoir increases with

the number of coefficients).

Weight normalization. Due to the fact that the approach depends on the IDCT mapping, it would be good to investigate in future works if a control of the weights can be beneficial. Probably, weight normalization after each generation can help to avoid neuron saturation, with additional computational cost to the method.

7. Conclusions and future work

In this paper, we added to the standard RC approach an evolutionary procedure for finding an optimal (in some sense) recurrent structure (reservoir matrix). Applying evolutionary methods to optimize recurrent structures seems to be a good strategy to avoid the well-known limitations presented in gradient-based methods. To reduce the searching space, we applied a compression of the reservoir weights, by moving them into the frequency domain using the DCT transform. Then, the reservoir weights were found by means of an evolutionary search.

We have analyzed the model using two very popular EAs: GAs and PSO. An empirical analysis over different datasets showed the potential of the idea. In general, we needed just a few evolutionary generations for achieving a higher accuracy than the standard ESN and its most popular modern variations. With respect to the initial version [25], we integrated many results. We improved the global procedure using the PSO approach. PSO is more stable than GA (with respect to the number of Fourier coefficients to use), and achieves similar accuracy than GA with a smaller number of parameters and of iterations. We also explored the impact of the initialization of the search in the Fourier space, and proposed specific ways of starting this exploration. In addition, we analyzed the evolution of the reservoirs and the stability issues.

We believe that our technique opens new possibilities for improving these tools, because we obtained better results from moving to the Fourier space leading to a dimensionality reduction, or in the EA side, by choosing a better optimization method. The exploration of the proposed approach in hierarchical architectures is an obvious research direction to be investigated. To compare our results with other indirect encoding approaches deserves also attention in the near future.

Acknowledgments

This work was supported by the GACR-Czech Science Foundation project No. 21-33574K “Lifelong Machine Learning on Data Streams”, and it was also supported by the ClimateDL project (code 22-CLIMAT-02) belonging to the Climate AmSud programme.

References

- [1] J. Schmidhuber, D. Wierstra, M. Gagliolo, F. Gomez, Training Recurrent Networks by Evolino, *Neural Networks* 19 (2007) 757–779.
- [2] M. Lukoševičius, H. Jaeger, Reservoir Computing Approaches to Recurrent Neural Network Training, *Computer Science Review* 3 (2009) 127–149. doi:10.1016/j.cosrev.2009.03.005.
- [3] J. Martens, Learning Recurrent Neural Networks with Hessian-Free Optimization, in: *Proceeding of the 28th International Conference on Machine Learning*, JMLR.org, Bellevue, WA, USA, 2011, pp. 1033–1040.
- [4] G. Tanaka, T. Yamane, J. B. Héroux, R. Nakane, N. Kanazawa, S. Takeda, H. Numata, D. Nakano, A. Hirose, Recent advances in physical reservoir computing: A review, *Neural Networks* 115 (2019) 100–123. doi:10.1016/j.neunet.2019.03.005.
- [5] L. Gonon, J.-P. Ortega, Fading memory echo state networks are universal, *Neural Networks* 138 (2021) 10–13. doi:10.1016/j.neunet.2021.01.025.
- [6] J. Schmidhuber, Deep learning in neural networks: An overview, *Neural Networks* 61 (2015) 85–117.
- [7] D. Verstraeten, B. Schrauwen, M. D’Haene, D. Stroobandt, An experimental unification of reservoir computing methods, *Neural Networks* 20 (2007) 287–289.
- [8] T. Elsken, J. H. Metzen, F. Hutter, Neural Architecture Search: A Survey, *Journal of Machine Learning Research* 20 (2019) 1–21.
- [9] A. T. Sergio, T. B. Luderger, PSO for Reservoir Computing Optimization, in: *Artificial Neural Networks and Machine Learning (ICANN’2012)*, volume 7552 of *Lecture Notes in Computer Science*, Springer Berlin Heidelberg, 2012, pp. 685–692.

- [10] S. Zhong, X. Xie, L. Lin, F. Wang, Genetic algorithm optimized double-reservoir echo state network for multi-regime time series prediction, *Neurocomputing* 238 (2017) 191–204. doi:10.1016/j.neucom.2017.01.053.
- [11] A. A. Ferreira, T. B. Ludermir, R. R. B. De Aquino, An Approach to Reservoir Computing Design and Training, *Expert Syst. Appl.* 40 (2013) 4172–4182. doi:10.1016/j.eswa.2013.01.029.
- [12] S. Basterrech, E. Alba, V. Snášel, An Experimental Analysis of the Echo State Network Initialization Using the Particle Swarm Optimization, in: *Nature and Biologically Inspired Computing (NaBIC)*, 2014 Sixth World Congress on, 2014, pp. 214–219. doi:10.1109/NaBIC.2014.6921880.
- [13] K. C. Chatzidimitriou, P. A. Mitkas, A NEAT Way for Evolving Echo State Networks, in: *Proceedings of the 2010 Conference on ECAI 2010: 19th European Conference on Artificial Intelligence*, IOS Press, NLD, 2010, pp. 909–914.
- [14] F. Matzner, Neuroevolution on the edge of chaos, in: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO’17*, ACM, New York, NY, USA, 2017, pp. 465–472. doi:10.1145/3071178.3071292.
- [15] Q. Ma, L. Shen, G. W. Cottrell, DeePr-ESN: A deep projection-encoding Echo State Network, *Information Sciences* 511 (2020) 152–171. doi:10.1016/j.ins.2019.09.049.
- [16] M. Dale, Neuroevolution of hierarchical reservoir computers, in: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO ’18*, ACM, New York, NY, USA, 2018, pp. 410–417. doi:10.1145/3205455.3205520.
- [17] C. Gallicchio, A. Micheli, L. Pedrelli, Deep reservoir computing: A critical experimental analysis, *Neurocomputing* 268 (2017) 87–99. doi:10.1016/j.neucom.2016.12.089.
- [18] A. Racca, L. Magri, Robust Optimization and Validation of Echo State Networks for learning chaotic dynamics, *Neural Networks* 142 (2021) 252–268. doi:10.1016/j.neunet.2021.05.004.
- [19] M. Lukoševičius, A. Uselis, Efficient cross-validation of echo state networks, in: I. V. Tetko, V. Kůrková, P. Karpov, F. Theis (Eds.), *Artificial Neural Networks and Machine Learning – ICANN 2019: Workshop and Special Sessions*, Springer International Publishing, 2019, p. 121–133. doi:10.1007/978-3-030-30493-5_12.

- [20] B. Schrauwen, M. Wardermann, D. Verstraeten, J. J. Steil, D. Stroobandt, Improving Reservoirs using Intrinsic Plasticity, *Neurocomputing* 71 (2008) 1159–1171.
- [21] A. Rodan, P. Tiño, Simple deterministically constructed cycle reservoirs with regular jumps, *Neural Computation* 24 (2012) 1822–1852. doi:10.1162/NECO_a.00297.
- [22] J. Koutník, G. Cuccu, J. Schmidhuber, F. Gomez, Evolving large-scale neural networks for vision-based reinforcement learning, in: *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation*, Association for Computing Machinery, New York, NY, USA, 2013, pp. 6–10.
- [23] S. Basterrech, G. Rubino, Evolving Reservoir Weights in the Frequency Domain, in: *2021 Genetic and Evolutionary Computation Conference Companion (GECCO '21 Companion)*, ACM, 2021.
- [24] J. Koutník, F. Gomez, J. Schmidhuber, Evolving neural networks in compressed weight space, in: *Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation, GECCO '10*, Association for Computing Machinery, New York, NY, USA, 2010, pp. 619–626. doi:10.1145/1830483.1830596.
- [25] S. Basterrech, G. Rubino, Evolutionary Echo State Network: evolving reservoirs in the Fourier space, in: *2022 IEEE International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8. doi:10.1109/IJCNN55064.2022.9892892.
- [26] M. Clerc, J. Kennedy, The particle swarm - explosion, stability, and convergence in a multidimensional complex space, *Evolutionary Computation, IEEE Transactions on* 6 (2002) 58–73. doi:10.1109/4235.985692.
- [27] A. Engelbrecht, *Computational Intelligence: An Introduction*, second edition ed., Wiley, New York, NY, USA, 2007.
- [28] H. Jaeger, H. Haas, Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication, *Science* 304 (2004) 78–80. doi:10.1126/science.1091277.
- [29] A. Rodan, P. Tiño, Minimum Complexity Echo State Network, *IEEE Transactions on Neural Networks* 22 (2011) 131–144. doi:10.1109/TNN.2010.2089641.

- [30] C. Gallicchio, A. Micheli, Architectural and Markovian factors of Echo State Networks, *Neural Networks* 24 (2011) 440–456. doi:10.1016/j.neunet.2011.02.002.
- [31] S. Basterrech, Empirical analysis of the necessary and sufficient conditions of the echo state property, in: *International Joint Conference on Neural Networks, IJCNN’17*, IEEE Press, Anchorage, AK, USA, 2017, pp. 888–896. doi:10.1109/IJCNN.2017.7965946.
- [32] J. B. Butcher, D. Verstraeten, B. Schrauwen, C. R. Day, P. W. Haycock, Reservoir Computing and Extreme Learning Machines for Non-linear Time-series Data Analysis, *Neural Networks* 38 (2013) 76–89.
- [33] SILSO World Data Center, The international sunspot number, *International Sunspot Number Monthly Bulletin and online catalogue (1749-2021)*. Available from: <http://www.sidc.be/silso/>.
- [34] A. Hart, J. Hook, J. Dawes, Embedding and approximation theorems for echo state networks, *Neural Networks* 128 (2020) 234 – 247. doi:10.1016/j.neunet.2020.05.013.
- [35] H. Cisneros, J. Sivic, T. Mikolov, Benchmarking Learning Efficiency in Deep Reservoir Computing, in: *Conference on Lifelong Learning Agents*, 2022.
- [36] R. Pascanu, T. Mikolov, Y. Bengio, On the difficulty of training recurrent neural networks, *Proceedings of the 30th International Conference on Machine Learning* 28 (2013) 37–48.
- [37] K. O. Stanley, J. Clune, J. Lehman, R. Miikkulainen, Designing neural networks through neuroevolution, *Nature Machine Learning* 1 (2019) 24–35.
- [38] D. Floreano, P. Dürri, C. Mattiussi, Neuroevolution: from architectures to learning, *Evolutionary Intelligence* 1 (2008) 47–62. doi:10.1007/s12065-007-0002-4.
- [39] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, J. Clune, Deep Neuroevolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks for Reinforcement Learning, *Arxiv abs/1712.06567* (2017). URL: <http://arxiv.org/abs/1712.06567>.
- [40] A. Rawal, R. Miikkulainen, Evolving Deep LSTM-based Memory networks using an Information Maximization Objective, in: *Genetic and Evolutionary Computation Conference (GECCO 2016)*, Colorado, USA, 2016.

- [41] K. O. Stanley, R. Miikkulainen, Evolving neural networks through augmenting topologies, *Evolutionary Computation* 10 (2002) 99–127. doi:10.1162/106365602320169811.
- [42] K. O. Stanley, D. B. D’Ambrosio, J. Gauci, A hypercube-based encoding for evolving large-scale neural networks, *Artificial Life* 15 (2009) 185–212. doi:10.1162/artl.2009.15.2.15202.
- [43] C. Gallicchio, A. Micheli, Reservoir topology in deep echo state networks, in: I. V. Tetko, V. Kůrková, P. Karpov, F. Theis (Eds.), *ICANN’2019*, Springer International Publishing, Cham, 2019, pp. 62–75.
- [44] S. Risi, K. O. Stanley, Deep Neuroevolution of Recurrent and Discrete World Models, in: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO’19*, Association for Computing Machinery, New York, NY, USA, 2019, pp. 456–462. doi:10.1145/3321707.3321817.
- [45] J. Lehman, J. Chen, J. Clune, K. O. Stanley, Safe mutations for deep and recurrent neural networks through output gradients, in: *GECCO’18*, Association for Computing Machinery, New York, NY, USA, 2018, pp. 117–124. doi:10.1145/3205455.3205473.
- [46] T. Uriot, D. Izzo, Safe Crossover of Neural Networks through Neuron Alignment, in: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference, GECCO ’20*, Association for Computing Machinery, New York, NY, USA, 2020, pp. 435–443. doi:10.1145/3377930.3390197.
- [47] R. Miikkulainen, J. Liang, E. Meyerson, A. Rawal, D. Fink, O. Francon, B. Raju, H. Shahrzad, A. Navruzian, N. Duffy, B. Hodjat, Chapter 15 - evolving deep neural networks, in: R. Kozma, C. Alippi, Y. Choe, F. C. Morabito (Eds.), *Artificial Intelligence in the Age of Neural Networks and Brain Computing*, Academic Press, 2019, pp. 293–312. doi:10.1016/B978-0-12-815480-9.00015-3.
- [48] Y. Li, F. Li, PSO-based growing echo state network, *Applied Soft Computing* 85 (2019) 105774. doi:10.1016/j.asoc.2019.105774.
- [49] H. Jaeger, The “echo state” approach to analysing and training recurrent neural networks, Technical Report 148, German National Research Center for Information Technology, 2001.
- [50] M. Lukoševičius, A Practical Guide to Applying Echo State Networks, in: G. Montavon, G. Orr, K.-R. Müller (Eds.), *Neural Networks: Tricks of the*

Trade, volume 7700 of *Lecture Notes in Computer Science*, Springer Berlin Heidelberg, 2012, pp. 659–686. doi:10.1007/978-3-642-35289-8_36.

- [51] C. Gallicchio, A. Micheli, Echo state property of deep reservoir computing networks, *Cognitive Computation* 9 (2017) 337–350. doi:10.1007/s12559-017-9461-9.
- [52] X. He, T. Liu, F. Hadaeghi, H. Jaeger, Reservoir Transfer on Analog Neuromorphic Hardware, in: 2019 9th International IEEE/EMBS Conference on Neural Engineering (NER), 2019, pp. 1234–1238. doi:10.1109/NER.2019.8716891.
- [53] S. Wang, et al., Echo state graph neural networks with analogue random resistive memory arrays, *Nature Machine Intelligence* (2023) 104–113. doi:10.1038/s42256-023-00609-5.
- [54] G. Wainrib, M. N. Galtier, A local Echo State Property through the largest Lyapunov exponent, *Neural Networks* 76 (2016) 39–45.
- [55] L. Gonon, L. Grigoryeva, J.-P. Ortega, Risk bounds for reservoir computing, *The Journal of Machine Learning Research* 21 (2020) 9684–9744.
- [56] W. Maass, P. Joshi, E. Sontag, Computational Aspects of Feedback in Neural Circuits, *PLoS Computational Biology* (2007).
- [57] R. F. Reinhart, J. J. Steil, Regularization and stability in reservoir networks with output feedback, *Neurocomputing* 90 (2012).
- [58] H. Jaeger, M. Lukoševičius, D. Popovici, U. Siewert, Optimization and applications of Echo State Networks with leaky-integrator neurons, *Neural Networks* 20 (2007) 335–352.
- [59] K. Irie, J. Schmidhuber, Training and Generating Neural Networks in Compressed Weight Space, in: International Conference in Learning Representations (ICLR), Neural Compression Workshop, 2021.
- [60] A. Ferreira, T. Ludermir, Comparing evolutionary methods for reservoir computing pre-training, in: Neural Networks (IJCNN), The 2011 International Joint Conference on, 2011, pp. 283–290. doi:10.1109/IJCNN.2011.6033233.
- [61] N. F. Xue Y, Zhang Q, Self-Adaptive Particle Swarm Optimization-Based Echo State Network for Time Series Prediction, *International Journal of Neural Systems* (2021). doi:10.1142/S012906572150057X.

- [62] C. Sun, M. Song, S. Hong, H. Li, A Review of Designs and Applications of Echo State Networks, 2020. doi:10.48550/ARXIV.2012.02974.
- [63] J. Long, S. Zhang, C. Li, Evolving Deep Echo State Networks for Intelligent Fault Diagnosis, *IEEE Transactions on Industrial Informatics* 16 (2020) 4928–4937. doi:10.1109/TII.2019.2938884.
- [64] M. Clerc, *Particle Swarm Optimization*, ISTE, Wiley, 2010.
- [65] M. Elbes, S. Alzubi, T. Kanan, A. Al-Fuqaha, B. Hawashin, A survey on particle swarm optimization with emphasis on engineering and network applications, *Evolutionary Intelligence* 12 (2019). doi:10.1007/s12065-019-00210-z.
- [66] C. Sun, S. Hong, M. Song, Y. Zhou, Y. Sun, D. Cai, H. Li, TE-ESN: Time Encoding Echo State Network for Prediction Based on Irregularly Sampled Time Series Data, in: *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)*, 2021.
- [67] B. L. Shen, M. Wang, P. C. Yan, H. P. Yu, J. Song, C. J. Da, Stable and unstable regions of the lorenz system, *Scientific Reports* (2018). doi:10.1038/s41598-018-33010-z.
- [68] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, C. Gagné, DEAP: Evolutionary algorithms made easy, *Journal of Machine Learning Research* 13 (2012) 2171–2175.
- [69] S. Seabold, J. Perktold, *Statsmodels: Econometric and statistical modeling with python*, in: *9th Python in Science Conference*, 2010.
- [70] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, *Nature* 585 (2020) 357–362. doi:10.1038/s41586-020-2649-2.